

Software Requirements Specifications

Welding Gun Repair Tracker

Ryan Weber

Document contents:

1. [Problem Statement](#)
2. [Justification of Development Model](#)
3. [Data Collection](#)
4. [Client Information](#)
5. [User Characteristics](#)
6. [Scope / Requirements](#)
7. [Constraints](#)
8. [Operating Environment](#)
9. [Appendix](#)

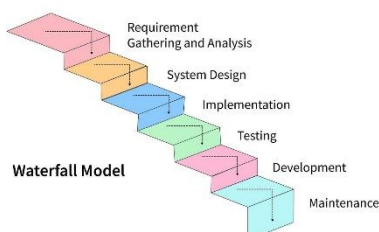
Problem Statement

My software will tackle the problem of inconsistent and lack of management with repairs of flame welding guns at a workplace. Currently they are not tracked and result in consequences such as using old or barely functioning welding guns, decreasing effectiveness of work and increasing cost for the company.

Justification of Development Model

3 Main Models:

Waterfall Model



The waterfall model is a linear development model, effective when client feedback isn't a priority, or when the client isn't a key part of the development. It is useful when the project needs to move in one direction without 2-way communication. Once a goal is reached, the project cannot be reversed. Its upsides are in its efficiency, and that project development can be completed quicker than other models, without the need for using time for looping back and forwards. Although, the waterfall model provides very little room for flexibility and adjustment, which are important when working closely with a client.

Agile Model



The agile model is a development model with repeating stages, which works effectively for projects with more client-developer feedback. It allows developers to make adjustments to their programs based off the feedback from the client. The agile model begins with an initial analysis stage, before cycling through design, development and evaluation stages, where the evaluation stage gathers input from the client. After enough cycles have been repeated, the project moves to the final evaluation stage, where a final assessment is made. The agile model suffers from a lack of efficiency, with a loss of time paying for a more thorough project, in greater accordance with the client's needs.

Spiral Model

Spiral model



The Spiral Model is a more abstract model, focusing on risks within the project's development. Similar to the agile model, it cycles through "phases," repetitively focusing on different stages of the development model. These stages are different from the waterfall and agile model, being planning, risk analysis, development and evaluation. The spiral model is more suited for larger projects with higher risk potential. It is less effective for small projects where all the risks are known beforehand.

My Choice

I have chosen the **agile development model**, as it suits the need for repetitive feedback with my client. My welding gun repair tracking program is for a relatively specific use, so frequent discussion with the client is necessary, allowing me to make the

appropriate modifications and to tailor the program closely to my client's needs. The agile model's feedback from client structure is also vital since I am not familiar with the functioning and use / maintenance of the welding guns. This is unlike the spiral model which provides less room for client feedback.

The agile model also provides flexibility, with me being able to change the design / program within the development-feedback stage. As building a program with an interface like the one needed for this is fairly new to me, the longer development, design and improvement cycle will allow for more room to learn and adjust, as opposed to the waterfall model which has a strict design stage before development begins. The waterfall model also does not leave any room for changing requirements and constraints.

My project will have a large range of possible features, meaning in the event of a time loss, the program can still be "completed" while in the event that there is more time than expected to develop, more features can be included. The agile model is better suited for these possible time restrictions as the design can be adjusted throughout the process.

Data Collection

Interview

Why I chose an interview:

I used an interview for data collection as interviews provide more comprehensive and more raw / true answers. As the amount of users for my program is relatively small, and interviews take a longer amount of time and planning, it is more useful for my case. Also, interviews provide opportunity for discussion and a 2-way conversation, which is more suited for me as this program's focus of tracking welding gun repairs is not a topic I am already knowledgeable about. An interview also lets me tailor questions based on answers as they happen, to get a deeper understanding of the program's requirements.

I interviewed my client.

I focused my interview questions around understanding the main goal for the software, how I can specifically achieve that, how users will prioritise different parts of the software, and what constraints I have due to the work environment the program will be running in.

The answers listed are notes I took while conducting the interview. Indentations signal questions asked in response to the client's answers.

Interview Questions and Answers:

- Why is there a need for software here? What are the flaws in the old system?
 - To tell which particular guns need the same part replacements, see how often problems arise, see how many parts are needed to buy for the future, predict cost, save cost, know the cost of repairs compared to buying guns, seeing how long they last.
 - Would it improve efficiency?
 - It would tell what guns are on the verge of breaking and help in predicting **patterns** on parts and how long they last. Ensures preventative maintenance can be done, to prevent breaking.
- What features would be useful in this program?
 - Click on gun and get information on every time it's been repaired, timeline, parts used in repair.

- Click on parts list and viewing the parts and part numbers, scroll through.
 - Graphs to see which guns are being repaired the most, which guns stand out. Use a bar graph, so you can see if there is a faulty gun. Investigate if a part is a problem, use data to see what the problem is, tell if the problem is with a gun or part.
-
- When and how often will the program be used? How many people will be using it?
 - I will use it once a week, sometimes 2 times in a day, varies, but on average once a week.
 - 2 people who have been trained specifically for repairs.
-
- What specific things need to be recorded when documenting a repair?
 - Date (must)
 - Gun serial number
 - Issue, problem
 - Repair, (what was done to repair the issue, can be null and filled out again later)
 - What parts were used
 - Repairer name
-
- What hardware will the program be running on?
 - Windows 10
 - Standard Mouse and standard full-size keyboard
 - Resolution – 1920*1080
-
- Are there any limitations to using the program in work, or any other constraints such as bad vision?
 - Rough and dirty environment, mouse is on a rough surface, making hard to control it, bad for pressing small buttons.
 - No vision blockage or sunshine
 - Medium motor skills
 - Medium detail, need to be seen from a distance (sometimes a meter away).
-
- Would choice of colour be important?
 - Simple colors. There could possibly be other people being trained to repair guns and wanting to use the program.
-
- Would intuitiveness be important?
 - Yes, simple for new people to learn, self-explanatory.
 - Not over complicated, just information, dates etc... **Simplicity.**

I also asked more questions on relevant topics that were not planned before the interview.

- How should the program handle awaiting repairs?
 - Show awaiting repairs with repair notes, giving cautions in the case that guns were not repaired, for other people to take note of. Include a tick-box for **completing** repairs, maybe waiting on parts. Have the ability to make comments. Colour; waiting on parts = yellow, waiting on repair = red, completed = green,

dropdown box for repair status, **filters**, irreparable, dead, graveyard – could help us figure out if we should hire, or buy guns.

- What should the design look like?
 - Tell how many guns, maybe use numbers for number of repairs. Scrollable list of guns.
 - Enter in different data types for parts, enter “nozzle” or “103”, int and string. **Versatile.**

Analysis of collected Data:

My interview allowed me to learn deeper about how my client wants the program to behave and look. I was able to find multiple important areas for development including:

- Design - The program will be used in rough environments, The monitor is viewed from a far distance, reducing visibility. The mouse used for the computers is on a rough surface, this means users will struggle to be accurate with clicks and buttons. This means some design choices have to be made:
 - Upscaling buttons – due to the low mouse accuracy, it would be helpful to upscale most, or important buttons so that they can be easily pressed. This increases both efficiency and effectiveness as users are less likely to miss-click, and can be more confident in pressing buttons quickly. It is a trade – off for screen space, meaning as a consequence less information can be displayed.
 - Using contrasting and appealing colours – A way to counter the far distance from the monitor and resulting reduced visibility is to use contrasting colours for buttons, making buttons more obvious and ensuring the user is aware of what is a button and what isn't. Another design choice influencing the colour of the solution would be having buttons change colour when hovered. This would provide feedback for the user, letting them know that the program is ready for their action. These colours would increase effectiveness, helping the user to not click on the wrong thing, and also increase efficiency by letting them tell quicker what is a button and what isn't.
- Range of Usability – I also importantly noted that new users must be able to use the program with minimal need for training, so the program must be intuitive. Ideally, new users can use the program with minimal training, while more experienced users can utilize the program to its fullest extent, allowing them to be more efficient and getting more value out of it. Design choices include:
 - Simplifying actions and text – using short words or symbols to show what buttons do, or what labels show. This would allow new users to intuitively use key functions.
 - Small info boxes – having small boxes describing the function of buttons and labels would allow new users to easily learn about the functionality of the program while not hindering experienced users. It is more effective than needing a manual, since information can be actively given through the program directly from each element.
- Effective ways to view information - In the interview it became apparent that a vast range of information about guns needs to be displayed, and accessed via a number of ways, including via each guns' menu, and by recent repairs. The interview emphasised that different users of the program need to be able to easily view and edit repairs made by other users. Since this program may be used by multiple people on a single device / instance, it is important that every person can easily see and interpret the actions of everyone else, so that they know what to focus working on in their work. Design choices include:
 - Page navigation – repairs should be able to be accessed through gun menus, and main repair menus. It should be easy for users to find a repair page they are looking for.
 - Colour indicators – Users should be able to see different repair statuses with colours or other visual means. This will help notify users of important issues which may be hard to communicate otherwise.
 - Comments – users should be able to leave comments on repairs to provide details to other repairers, either warning them about problems with a gun, stating what they have already done to try and repair it, or giving any other extra info.

- Viewing trends and patterns – One key takeaway from the interview is that it is important that the program can easily show trends in repairs / issues. This would allow users to predict a gun's life, giving them the option for preventative maintenance to be done on the guns before they break. Users can tell if it is better to buy or hire guns, ultimately saving money and time. Design choices include:
 - Issue indicator - Visually showing the amount of issues or repairs that a gun has ever had through a number, or graph displayed on the gun, clearly giving an indication of whether that gun is on the verge of breaking, or how damaged it is.
- Versatility – The program may be used by a range of workers who input data in non-expected forms. It is important that the solution recognises this and does not attempt processing without suitable types of data, improving effectiveness by preventing the program from crashing.
 - Non-restrictive data input - It is important that raw data is shown how it is inputted, and the solution tries not to restrict the users inputs, which would reduce efficiency. For example, if the user is prompted for the part number they used for a repair, they should be able to input “nozzle” (string) as well as “103” (number) and be accepted.

This interview was successful in finding the priorities and intentions of my client.

Survey

Why I chose a survey:

I used a survey to gather mainly numeric responses from users. My survey allows for efficient and easily analysable data, so I can get a better understanding of preferences and needs for different aspects for the program.

My survey's questions are centered around understanding the importance of different aspects of the software to the user. The written response questions also provide further insight as to what is prioritised by the users, and allow for any further important information.

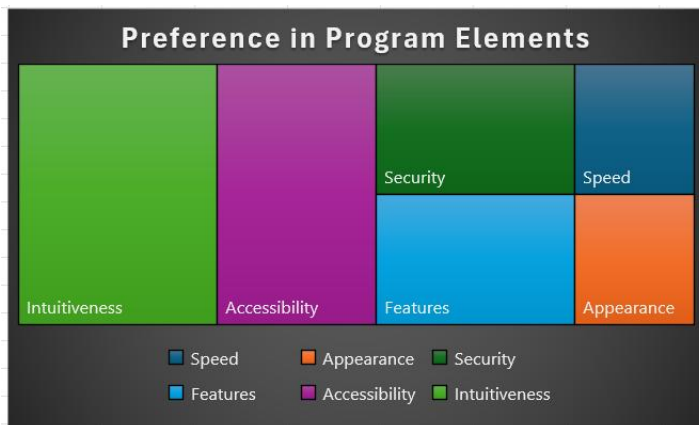
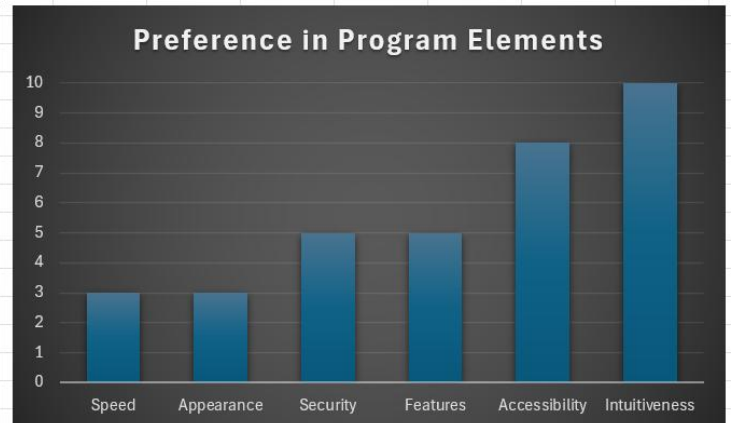
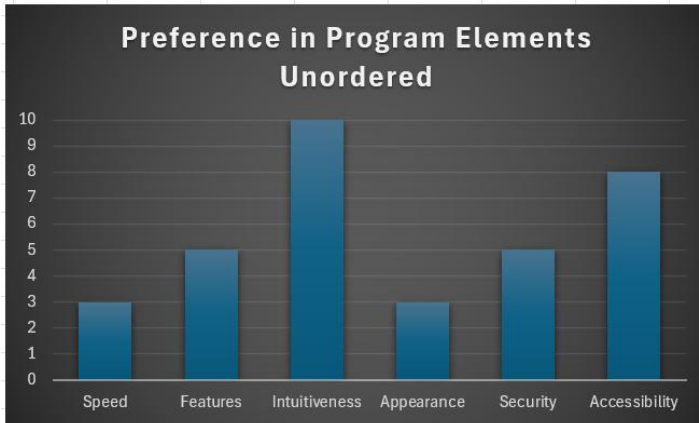
My client answered my survey.

Survey Questions and Responses:

Main

- How many times do you make a repair per week?
 - 1
- Rate importance of 1 - 10:
 - Speed (easy to use, loading times)
 - 3
 - Features
 - 5
 - Intuitiveness (know what to do)
 - 10
 - Appearance
 - 3
 - Security (not losing data, having backups)
 - 5
 - Accessibility (settings for font sizes and other visual elements)
 - 8
- Is there anything else important to take note of?

- Hopefully this data will encourage others to read back on notes and get insight as to common problems, and educate themselves on how to repair the guns, and know the general issues, using previous notes, making it easier for other people.
- This work is in the early stages, data will help others to learn how to repair, other users are inexperienced.



Analysis of collected Data:

The survey was shorter than the interview and mainly focused on quantitative info about the frequency of repairs and the importance of different aspects of the program, with an extended response section at the end. Although there was nothing to specifically measure how different aspects of the program would work, The survey still gave a good indication of what the user is looking for in terms of functionality and appearance on average.

Question 1:

How many times do you make a repair per week?

Answer 1:

1

This question was asked to get a gauge of how much data the program will be handling. It affects how data is displayed and processed, as for example, with a high frequency of repairs, search functions may be slower and so may have to be programmed differently. The program will be used once a week, but more users will also start using the program in the future (as found out in the interview). It is feasible to use a dropdown box for names, and display a few recent repairs or guns on the screen at a time.

Question 2:

Rate importance of 1 - 10:

- Speed (easy to use, loading times)
- Features
- Intuitiveness (know what to do)
- Appearance
- Security (not losing data, having backups)
- Accessibility (settings for font sizes and other visual elements)

Answer 2:

Aspect rankings:

- | | |
|----------------------------|-------|
| 1. Intuitiveness (10) | ***** |
| 2. Accessibility (8) | ***** |
| 3. Features / Security (5) | ***** |
| 4. Appearance / Speed (3) | *** |

“Intuitiveness” was the highest-ranking aspect. This means it is important for the program to be easy to understand for beginners. As “features” was ranked at half that (a five), it is shown that that the program should have working, intuitive and easy to use, reliable functions and features before anything more complex is added. Early development should focus on the main, simple functions such as adding and viewing repairs. More complex features can still be added later in the development, where the rest of the program has already been refined. “Accessibility” was ranked 2nd (an eight), and speed was ranked at a 2. This means it is worthwhile to trade effectiveness for efficiency in most cases, and development can focus more on having functions work properly and easily instead of quickly.

Answer 3:

- Hopefully this data will encourage others to read back on notes and get insight as to common problems, and educate themselves on how to repair the guns, and know the general issues, using previous notes, making it easier for other people.
- This work is in the early stages, data will help others to learn how to repair, other users are inexperienced.

The question’s answer focuses on the use of the program. It will be important to let users easily leave comments and other data for other users.

Observation

Why I chose to use observation:

Observing how users interact with the old system gives an unbiased exact measure of how the people track welding gun repairs. It gives data on how often repairs are documented, how many people document them, what the main problems are and what data is recorded about the welding guns.

Observation:

I am observing **the use of** a written sheet as an old method to track how people do repairs on welding guns.

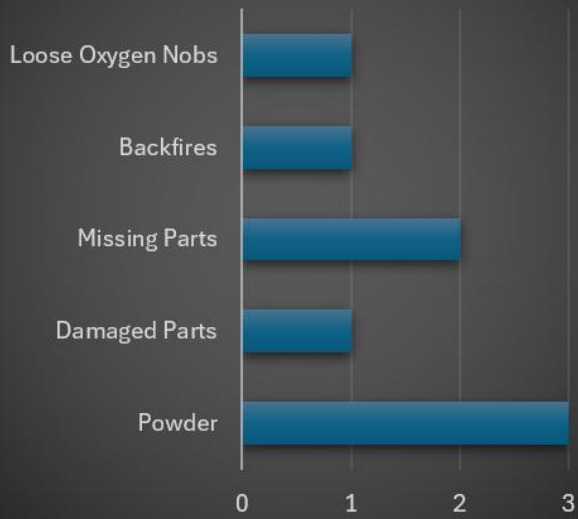
Written Sheet:

Spray gun repair sheet

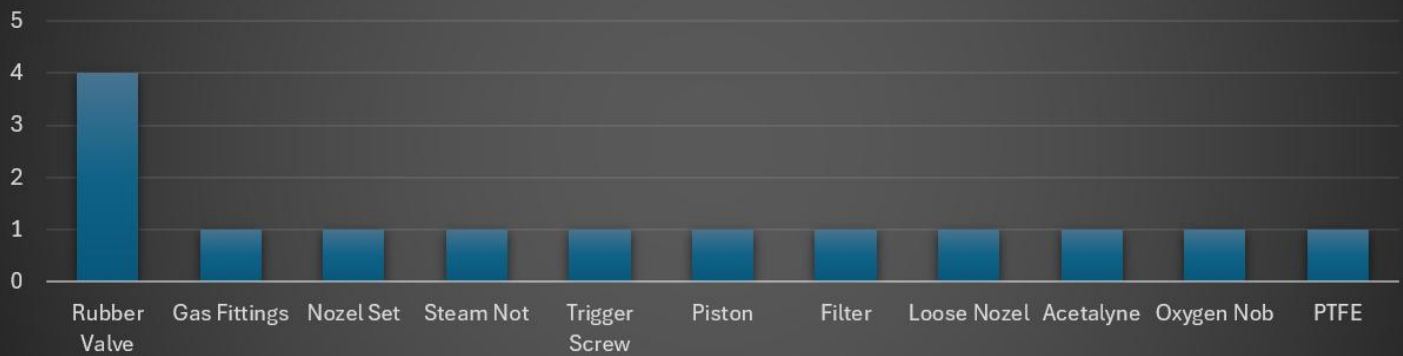
Date	Serial #	Problem	Resolution	Repairer	Parts #
14.2.24	K2264 #10	Powder not coming out	Contamination in dust container. replaced rubber powder feed valve	Andre	150
26.2.24	K2348 #11	Powder not coming out	replaced rubber powder feed valve	Andre	150
11.3.24	i4021 #6	Unknown - missing parts - damaged / worn out parts.	replaced rubber feed valve. replaced gas fittings replaced nozzle set replaced stem not set replaced PTFE Washer	Andre	150 158 200 198 153 201 203
29.3.24	K3088 #3	missing trigger screw	replaced missing trigger screw	Takumi	145
21.4.24	K2264 #10	trigger not springing back (piston has no 'springs').	piston spring o.k. Piston bent & straightened dislodged piston cross pin, 4mm long.	Andre	145
21.4.24	K3382	no powder back fires	replaced rubber powder feed valve. replaced 3ml + Lrg sintered filters. loose nozzle	Andre	150 120 121
4.5.24	K3642 #	loose oxygen knob	fixed knob oxygen + Acetylene both loose.	Andre	

Different Representation of Data:

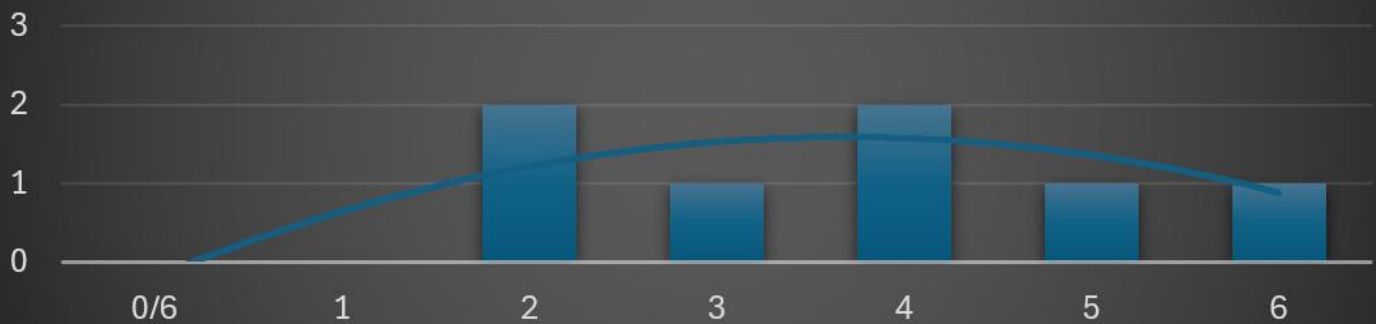
Problem Frequency



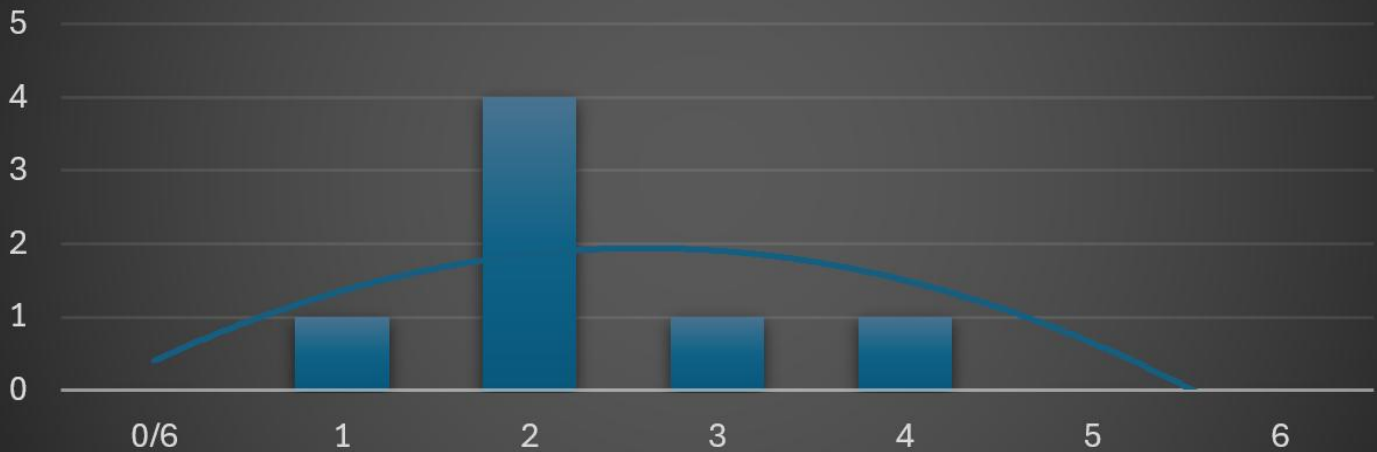
Fix Frequency



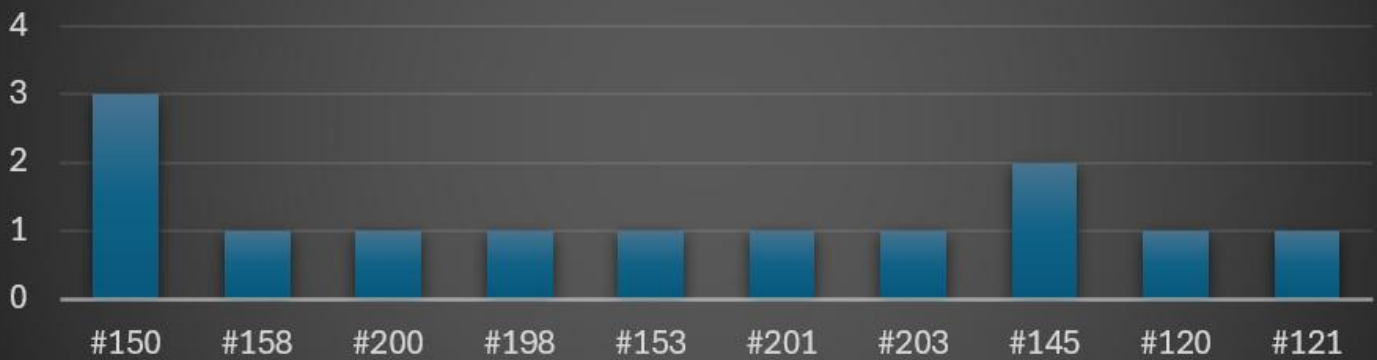
Fix Line Count Frequency



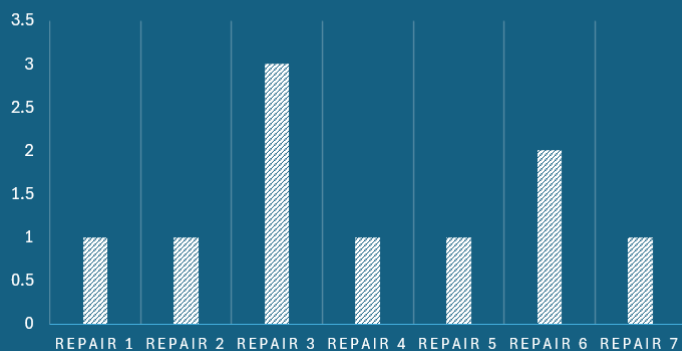
Issue Line Count Frequency



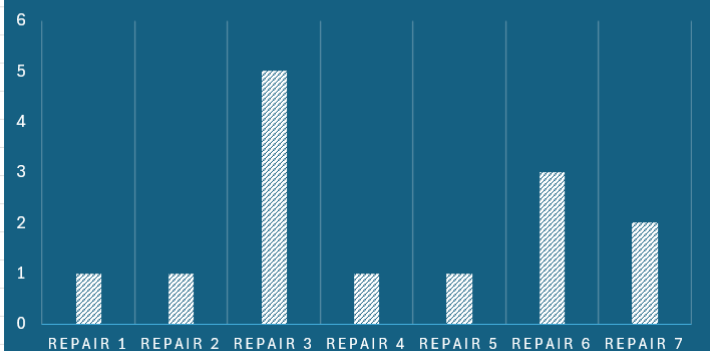
Part Frequency



ISSUES PER REPAIR



FIXES PER REPAIR



Analysis of collected Data:

Sheet Box Titles:

The sheet shows the date, gun serial number, repair notes (such as what the problem was, and what was done to fix it), the repairer's name, and the index number of the parts used in the repair. These are the key things that the program needs to be able to store in a repair. Storing the date can be automated, and extra notes can be entered with a text box, while everything else including the repair problem / resolution boxes can use a dropdown box for faster entry.

As there are minimal boxes for data entry in the sheet, the program should follow the same idea and ensure that the minimum added repairs are kept simple.

When adding a repair, workers use a picture to determine the part numbers of the parts used in the repairs. It would be greatly beneficial to include the parts visual somewhere in the program when adding a repair. Not having this in the current system is a big hindrance as viewing the part numbers requires looking at a picture in another place, which may not always be on hand and takes time to look at.

Using a written sheet in general is not ideal as it could be lost, and it isn't easy to change data, share it, or add new information to the repairs.

Sheet Entries:

The sheet entries are repetitive, with multiple "No Powder" and "Replaced Rubber Feed Valve" entries. These entries take time to write and can be made much more efficient with a dropdown box mentioned before. As some repairs are done multiple times in a day, it might be beneficial to save some input data, such as the repairers' names.

It is important to note that some entries such as the serial number are left blank. The program should ensure that any user can do this, and it will still function properly.

Another observation is that some text boxes have barely enough room to record the data, so it may be an issue for any entries that may extend beyond these boxes. Therefore my program should allow entries (or extra notes) of any length.

Furthermore, many entries have multiple issues or fixes, that need to be all displayed on the same gun / repair item. This means the program cannot ONLY use a dropdown box to select a single entry. There needs to be enough visual space in the program to list all fixes and issues.

Client Information

Company

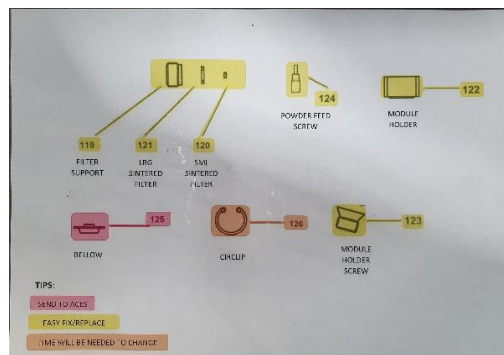
Business Name: Visy (Glass) <https://www.visy.com/products/glass>

Business Type: Large Business, manufacturing / packaging / recycling / logistics company

Client Name: Andre, worker in Visy

Visy's Vision: "Our vision is to be the global leader in creating sustainable packaging solutions for a better world"
<https://www.visy.com/about/visy>

Visy has many subgroups, one of which is glass bottle manufacturing, requiring glass bottle mould repairs and the use of flame welding guns.



Solution Audience

The audience for this solution is Visy workers who are repairing, using and maintaining flame welding guns for glass bottle mould manufacturing. The program is a tool which lets users track the repairs of the flame welding guns, and is required for their proper maintenance, and to save cost, time and parts by being able to interpret the status of different guns, and trends in their problems. It is a private solution and is specifically meant for this need.

The repair of flame welding guns is a small portion of glass bottle manufacturing for Visy and so has a small audience.

User Characteristics

- Age: Middle-aged
- No disabilities
- Experienced in manufacturing / fitting and turning. Past experience with using a written system, knowledgeable about requirements and use.
- Knowledgeable and experienced with computers / technology.

Scope / Requirements

Functional Requirements

- View repairs tab
 - Users can view a list of all repairs.
- Log repairs
 - Users can use a button to add a repair to the list of repairs.
- Entries (drop down box)
 - Users can add tags and details to repair entries by using dropdown boxes for each detail. Users can add multiple tags to each repair. Having these dropdown boxes will increase the efficiency of the program. Details being recorded will include problems, fixes, repairer name (to see if repairers need to improve on a specific repair), gun serial number, date (pre-entered but editable), new part numbers, and extra comments. The entry will also include non-editable info like the number of times which that specific gun has been repaired.
- Typed entries
 - The user can manually type their own entry in the event that it does not exist as an option in the dropdown box, ensuring system effectiveness.
- Sort by different categories
 - The repairs list and gun list can be sorted by most recent, most damaged, most times repaired, cost, frequency (repairs over time used), etc. Some guns may need preventative maintenance to prevent breakdowns, so it is important to be able to quickly see trends and get information. This will also help users know what parts are used a lot and may need to be swapped out, and will help users learn how to properly maintain the guns.
- Letting users write comments in repairs

- Being able to write comments in repairs lets users input unique entries such as new problems, or an uncommon error / mistake that can be viewed later.
- Display parts list with numbers
 - When entering a repair, users can view an image showing the part number of every part to a welding gun, allowing them to easily know what the part number is of the new part so they can input it into the repair (see “Entries (drop down box)”). This increases the program’s usability and effectiveness.
- Manually adding tags to dropdown boxes
 - To increase the program’s self-maintenance, the program will let users add their own tags to the dropdown boxes in the details for each repair. This is very useful in the case of new users, or in case of new problems or repairs. This allows the users to have more manual access to the functionality of the program. There will also be a shortcut for this when entering a custom typed entry as stated in “Typed entries”.
- Display graphs and UI on each repair
 - The program will show easily readable visuals on each gun and repair for accessibility and ease of reading and getting information. This can be things like showing the number of repairs on each gun with filled-in squares, a bar, or text.
- Click repairs to view info and edit
 - Users will be able to open a repair menu by clicking on a repair, letting them view more details, and editing it. This helps the program’s maintenance and adaptiveness, and is useful in case of entry errors or other issues.
- Write and read data from data to csv file
 - The program will write data to a csv in a readable format, which can be printed, making the program more effective. The program will read a csv to create the list of all repairs when the program is opened, and modify it when edits are made.
- Filter repairs
 - The user can filter repairs or welding guns, only showing ones with certain characteristics. These can include repairs made in a certain date range, types of repairs, types of problems, users who did the repair, etc.
- Colour by attributes
 - Users can change which colours are used to display different details, and what details the colours are displaying. This ensures the program is attractive, and that users can easily see data about any repair. Details represented by colour could include serial number, user who made repair, times gun has been repaired, age of gun, etc.
- Deleting repairs
 - In the case of a mistake, the user will be able to delete a repair, which will delete it from the csv.
- Gun menu tab
 - The Program will display a tab showing every welding gun, with the option to view each gun specifically by clicking on them.
- Show gun repair history
 - Each gun will display its repair history when the user is in the specific gun menu. The repair history of a gun will also be displayed in the menu of each repair after it is added.
- Search for repair or gun

- Users can type in serial numbers, dates or potentially other data to retrieve specific results, increasing efficiency.
- Add gun / send gun to graveyard
 - The program will have a “graveyard” where users can move welding guns to, meaning their repair data won’t be deleted, but they will be differently categorised in the gun tab with a different colour, or be sent to the bottom of the list. Repairs of that gun will also be shown differently.
- Recover guns from graveyard / delete gun from graveyard
 - The user can delete guns from the graveyard in the case that they were accidentally created or otherwise. Users can recover guns from the graveyard in the result of an accidental move, increasing the program’s reliability.
- Access and change settings
 - The user can access a settings screen with many settings, changing how the program displays and inputs data, giving the user more control over the program.
- Closing out of windows
 - The program will close windows and cancel any current repair entries when a close button is pressed. The program will also have a close button, and safely close in a force-shutdown.

Non-Functional Requirements

- Accessibility
 - Users will be able to change font sizes for ease of reading, change colours of items and menus, change colour attributes / meanings as designated for differentiating categories, and change the type of list items to types such as box or horizontal.
- Robustness
 - The program will not crash under stress which may be caused by extensive searches or sorting. The program will also not crash with any errors, or when modifying or changing the csv the program is writing to and reading from. This helps maintain the effectiveness of the solution.
- Self-Maintenance
 - Users will be able to add items to the dropdown menus for each repair detail, as well as being able to add new guns, remove guns, or send guns to the graveyard. This means the program can be self-maintained by the users, with a high level of manual access to its features.
- Usability and Security
 - Data will be stored in a csv file to let users easily print and view lists, allowing backups to be manually created through the program or through copying the csv, increasing the safety of the data in the case of data-loss. This ensures the effectiveness of the program is not lost. Data must be saved in the csv to not be lost.

Constraints

Economic

- Apart from the electrical costs to run and develop the program, the budget for development is \$0. No assets or code is being used from any source but me, the developer.

Legal

- The program must not contain any materials that violate any copyright laws, such as images, logos, videos, etc. This is under the Australian *Copyright Act 1968*.
- The program's code must not be taken from any external sources such as online websites.
- To maintain authenticity, all development should be solely done by me. This ensures the program is original, and no stolen or work possibly breaching copyright acts are used.

Hardware / Software

- The software must run on a windows 10 computer with 1920*1080 resolution. It cannot use inputs from sources other than a mouse and full-size keyboard. The program cannot use sound.

Usability

- The program must be usable for workers with all skills with technology.
- Ideally, the program should allow inexperienced workers to understand and intuitively use it, while more experienced workers should be able to get more use and efficiency out of the more complex features of the program.
- The program should include instructions, either in a separate screen, or when hovering over certain things, to assist users in learning how to use all features. The program is constrained by how difficult it is for new users to learn how to use it.

Operating Environment

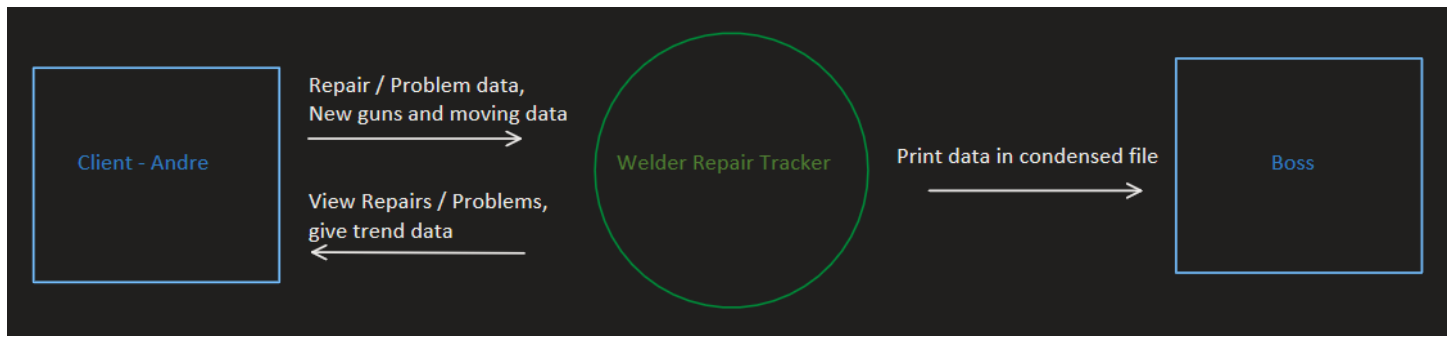
The operating environment is a fitting and turning workplace with rough conditions. Computers are positioned some distance away from the user.

Device – Windows 10 desktop with 1920*1080 resolution

Connection / Services – Internet access

Appendix

Context Diagram



Use Case Diagram



Data Flow Diagram

